SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: **Database Management Systems**
COURSE CODE: **CSA0508**

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)
Assessment Tool Weightage: 35%

Dr.G.Ayyappan

QUESTIONS
Total Marks: 50

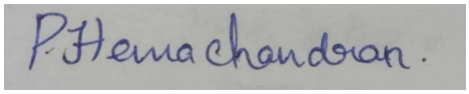
Q.No	Question	Bloom's Level	Marks
1	Apply relational algebra operations (σ, π, ∪, -, ×, ⋈) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	**5**
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	**5**
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT,	BL3 – Apply	**5**

	SUM, AVG, MAX, MIN) on a given Sales dataset.		
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	**5**
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	**5**
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	**5**
7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	**5**
8	Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.	BL3 – Apply	**5**
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	**5**
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	**5**

Code of Conduct:

I P.Hemachandran (192525440) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate;	Minor calculation errors	Frequent errors; incorrect	No valid calculations

		correct final answer		final answer	
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1. Relational Algebra Operations (σ, π, ∪, −, ×, ⋈)

Relational Algebra is a procedural query language used to retrieve data from relational databases. It consists of several operations that manipulate relations (tables) and produce new relations as results.

Relational Algebra Operators

Symbol (Name)	Example of Use
σ (Selection)	$\sigma_{salary>=85000}(instructor)$
	Return rows of the input relation that satisfy the predicate.
Π (Projection)	$\Pi_{ID,\ salary}(instructor)$
	Output specified attributes from all rows of the input relation. Remove duplicate tuples from the output.
⋈ (Natural Join)	*instructor* ⋈ *department*
	Output pairs of rows from the two input relations that have the same value on all attributes that have the same name.
× (Cartesian Product)	*instructor* × *department*
	Output all pairs of rows from the two input relations (regardless of whether or not they have the same values on common attributes)
∪ (Union)	$\Pi_{name}(instructor) \cup \Pi_{name}(student)$
	Output the union of tuples from the two input relations.

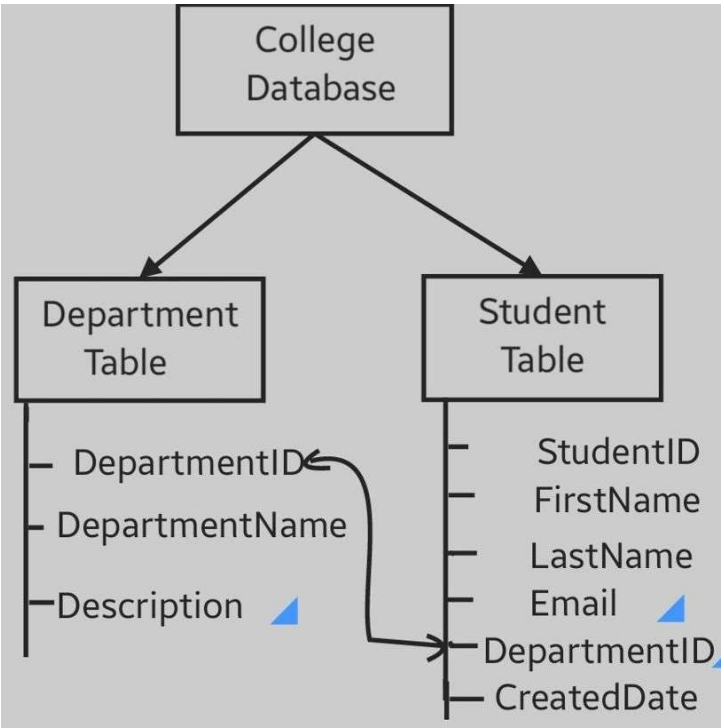
- **Selection (σ):** The selection operation retrieves rows (tuples) from a relation that satisfy a specified condition. For example, selecting employees whose salary is greater than ₹50,000.
- **Projection (π):** Projection retrieves specific columns (attributes) from a relation while removing duplicate values. For example, displaying only employee names.
- **Union (∪):** The union operation combines the tuples from two compatible relations and removes duplicate tuples. Both relations must have the same number of attributes and compatible data types.
- **Difference (−):** Difference returns tuples that are present in one relation but not in another. It is useful for finding records that do not satisfy certain conditions.
- **Cartesian Product (×):** The Cartesian product combines every tuple of one relation with every tuple of another relation, producing all possible combinations.
- **Join (⋈):** Join combines tuples from two relations based on a common attribute. It is commonly used to retrieve related information stored in different tables, such as employee details along with department names.

2. SQL DDL Statements

Data Definition Language (DDL) is used to define and manage the structure of database objects such as tables, indexes, and constraints.

The **STUDENT** table contains the student ID, student name, date of birth, and department ID. The **DEPARTMENT** table contains the department ID and department name.

Appropriate constraints include:



- **PRIMARY KEY:** Ensures each record is unique.
- **NOT NULL:** Prevents NULL values.
- **UNIQUE:** Prevents duplicate values.
- **FOREIGN KEY:** Maintains referential integrity by linking the STUDENT table to the DEPARTMENT table.

These constraints improve data integrity and consistency in the database.

3. GROUP BY, HAVING, and Aggregate Functions

Aggregate functions perform calculations on multiple rows and return a single value.

- **COUNT():** Returns the total number of records.
- **SUM():** Calculates the total of a numeric column.
- **AVG():** Finds the average value.
- **MAX():** Returns the highest value.
- **MIN():** Returns the lowest value.

The **GROUP BY** clause groups rows having the same values into summary rows. It is generally used with aggregate functions.

The **HAVING** clause filters grouped records after the GROUP BY operation. Unlike the WHERE clause, HAVING is used to specify conditions on groups.

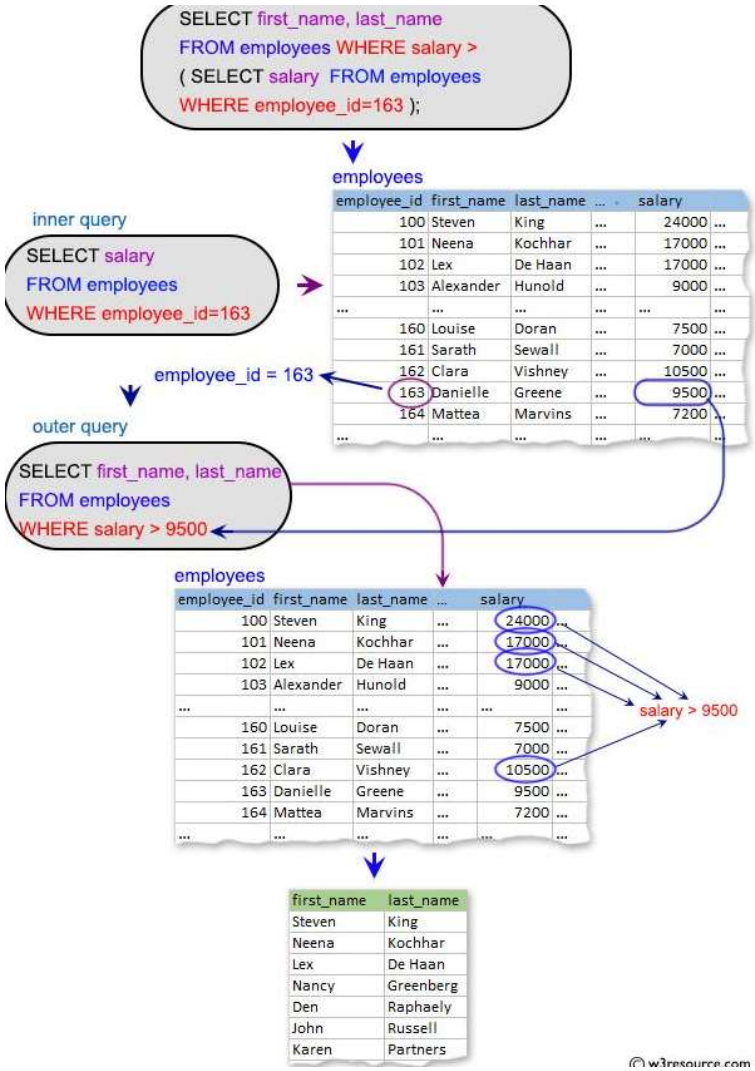
For example, in a Sales table, GROUP BY can calculate the total sales for each product, while HAVING can display only those products whose total sales exceed a specified amount.

4. Correlated Sub-query

A correlated sub-query is a sub-query that depends on the outer query for its execution. It is executed once for every row processed by the outer query.

For example, to find employees earning more than the average salary of their own department, the sub-query calculates the average salary for each department and compares it with the salary of each employee.

Correlated sub-queries are commonly used for row-by-row comparisons and complex filtering conditions.



5. Types of JOIN Operations

A JOIN operation combines rows from two or more tables based on a related column.

Inner Join

Returns only the matching records from both tables. Records without matching values are excluded.

Left Outer Join

Returns all records from the left table and only the matching records from the right table. If no match exists, NULL values are displayed for the right table.

Right Outer Join

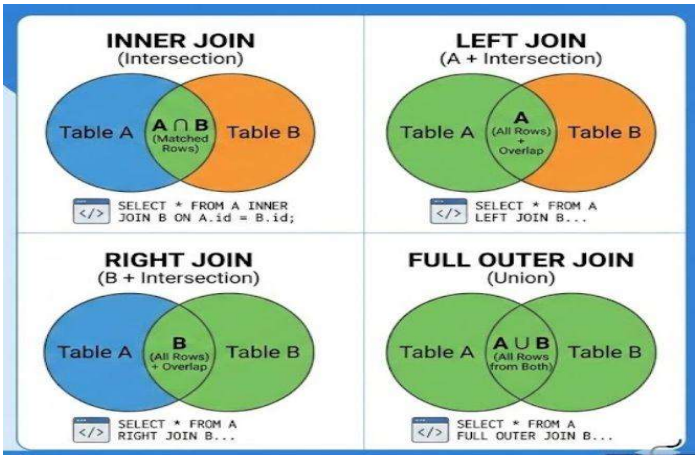
Returns all records from the right table and matching records from the left table. Unmatched rows from the left table contain NULL values.

Full Outer Join

Returns all matching and non-matching records from both tables. NULL values are displayed wherever no matching record exists.

Cross Join

Returns the Cartesian product of both tables by combining every row of one table with every row of the other table.



6. View – DeptSalaryView

A **View** is a virtual table created using a SELECT statement. It does not store data physically but displays data from one or more tables.

The view **DeptSalaryView** contains:

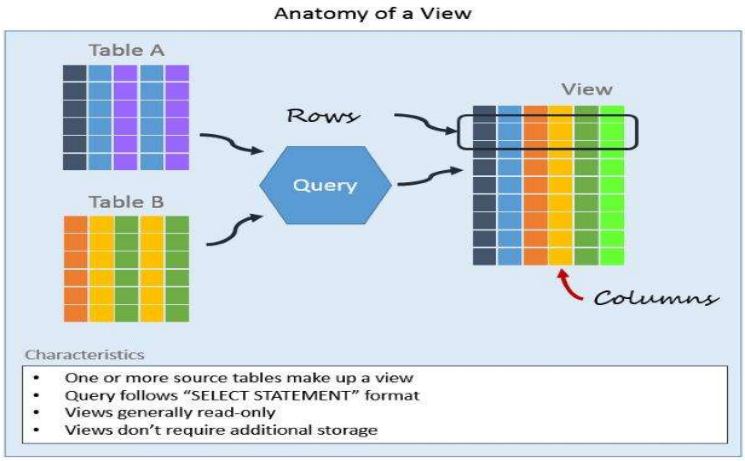
- Department Name
- Total Number of Employees

- Average Salary

Views provide the following advantages:

- Improve security by restricting access to specific columns.
- Simplify complex SQL queries.
- Improve code reusability.
- Provide data abstraction.

Queries can be executed on the view just like a normal table.

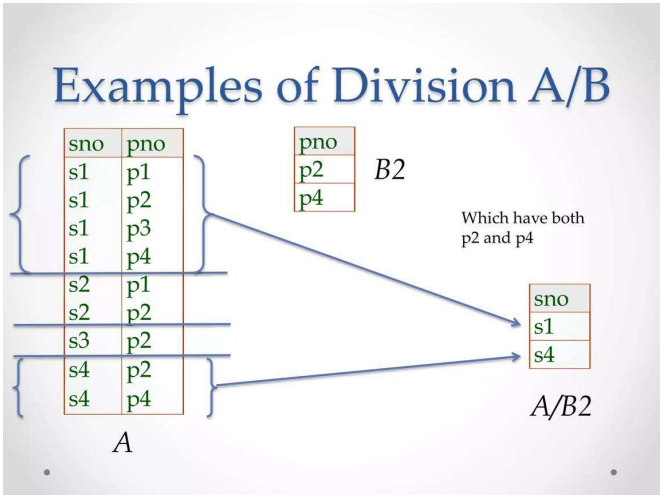


7. Relational Algebra Expression – Students Enrolled in All Courses

To find students enrolled in all available courses, the **Division (÷)** operator is used.

The Division operation returns those students who are associated with every course offered by the institution.

This query is useful in situations where "all" conditions need to be satisfied, such as finding students who completed every compulsory course.



8. SQL DML Statements

Data Manipulation Language (DML) is used to manipulate data stored in database tables.

The three major DML statements are:

INSERT

Adds new records into a table.

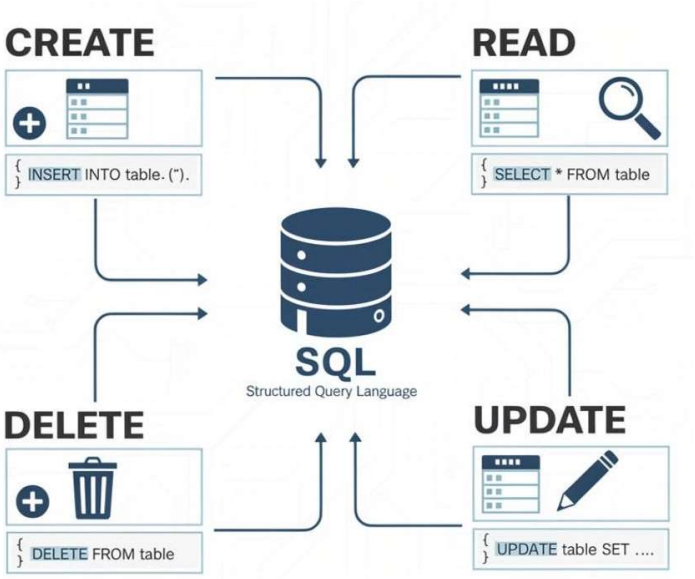
UPDATE

Modifies existing records in a table.

DELETE

Removes records from a table.

While performing DML operations, integrity constraints such as Primary Key, Foreign Key, Unique, and Not Null constraints ensure that the data remains valid and consistent.



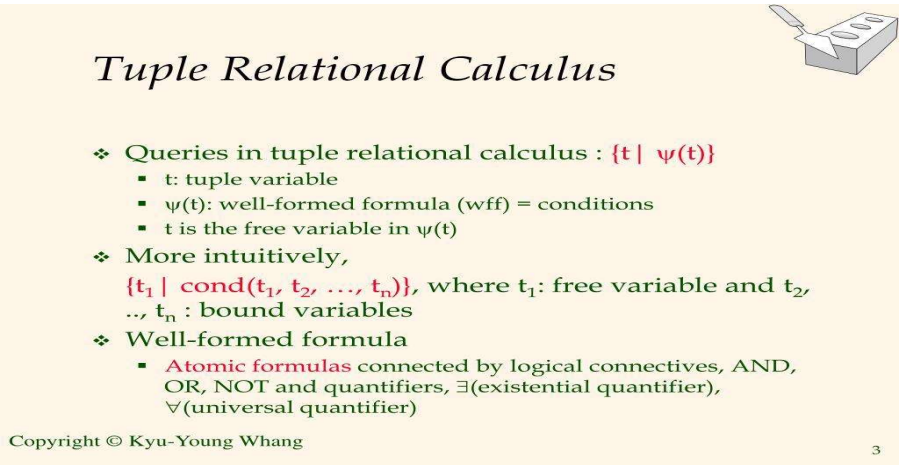
9. Tuple Relational Calculus (TRC)

Tuple Relational Calculus is a **non-procedural query language**. It specifies **what data is required rather than how to retrieve it**.

In Tuple Relational Calculus, variables represent tuples, and queries are expressed using logical predicates.

To find employees working in the **IT department** and earning more than **₹50,000**, the query specifies conditions that the employee must satisfy without describing the retrieval procedure.

TRC is based on first-order predicate logic and is equivalent in expressive power to Relational Algebra.



10. Airline Reservation System

An Airline Reservation System is a database application used to manage flight bookings, passenger information, and ticket reservations efficiently.

Main Tables

- **Passenger:** Stores passenger details such as Passenger ID, Name, Contact Number, and Address.
- **Flight:** Stores flight information including Flight ID, Flight Name, Source, Destination, and Departure Time.
- **Booking:** Maintains booking details such as Booking ID, Passenger ID, Flight ID, Seat Number, and Booking Date.
- **Payment:** Stores payment information such as Payment ID, Booking ID, Payment Mode, and Amount.

SQL Queries

The system demonstrates the use of various SQL concepts:

1. **JOIN Query:** Displays passenger names along with their booked flight details.
2. **Sub-query:** Finds passengers who paid more than a specified amount.
3. **Aggregate Query:** Counts the number of bookings for each flight using GROUP BY.
4. **View:** Creates a view showing flight names and total bookings.
5. **LEFT JOIN:** Displays all passengers, including those who have not yet made any bookings.

Advantages

- Reduces manual work.
- Ensures data accuracy.
- Supports quick ticket booking and cancellation.
- Provides efficient passenger and flight management.
- Improves security and maintains data consistency.